Language Agnostic AI Chatbot

Smart India Hackathon 2025 · Problem Statement 25104 · Smart Education Theme

Project Overview

This project is a multilingual education FAQ chatbot built for Smart India Hackathon 2025. It enables students to ask questions about college services in any Indian language — Hindi, Tamil, Telugu, Malayalam, Bengali, and more — and receive accurate answers in their own language. The system removes the language barrier from accessing institutional information.

Problem Statement

Indian educational institutions serve students who are more comfortable communicating in their native language. Existing FAQ systems are English-only, creating an accessibility gap. Problem Statement 25104 challenges teams to build AI tools that democratize access to educational information across language barriers.

Key Features

#	Feature	Description
1	Language Detection	Identifies user's language automatically using langdetect
2	IndicTrans Routing	Hindi queries routed through IndicTrans pipeline
3	Google Translate	All other languages handled via Google Translate API
4	English NLU	Normalizes all queries to English before intent classification
5	Rasa NLU	Classifies intent and extracts entities (slot filling)
6	FAQ Retrieval	55+ Q&A pairs across 12 education categories
7	Response Translation	Answers translated back to the user's language
8	Audit Logging	Every interaction logged to SQLite via Django ORM
9	Clean Chat UI	Responsive React + Tailwind interface with pipeline metadata
10	< 2s Response	End-to-end latency target: under 2 seconds on localhost

FAQ Categories

- Admissions
- Scholarships
- Hostel & Accommodation
- Attendance
- Examinations
- Library
- Fees
- Leave Policy
- Placements

● Faculty ● Canteen & Sports ● General / Administration

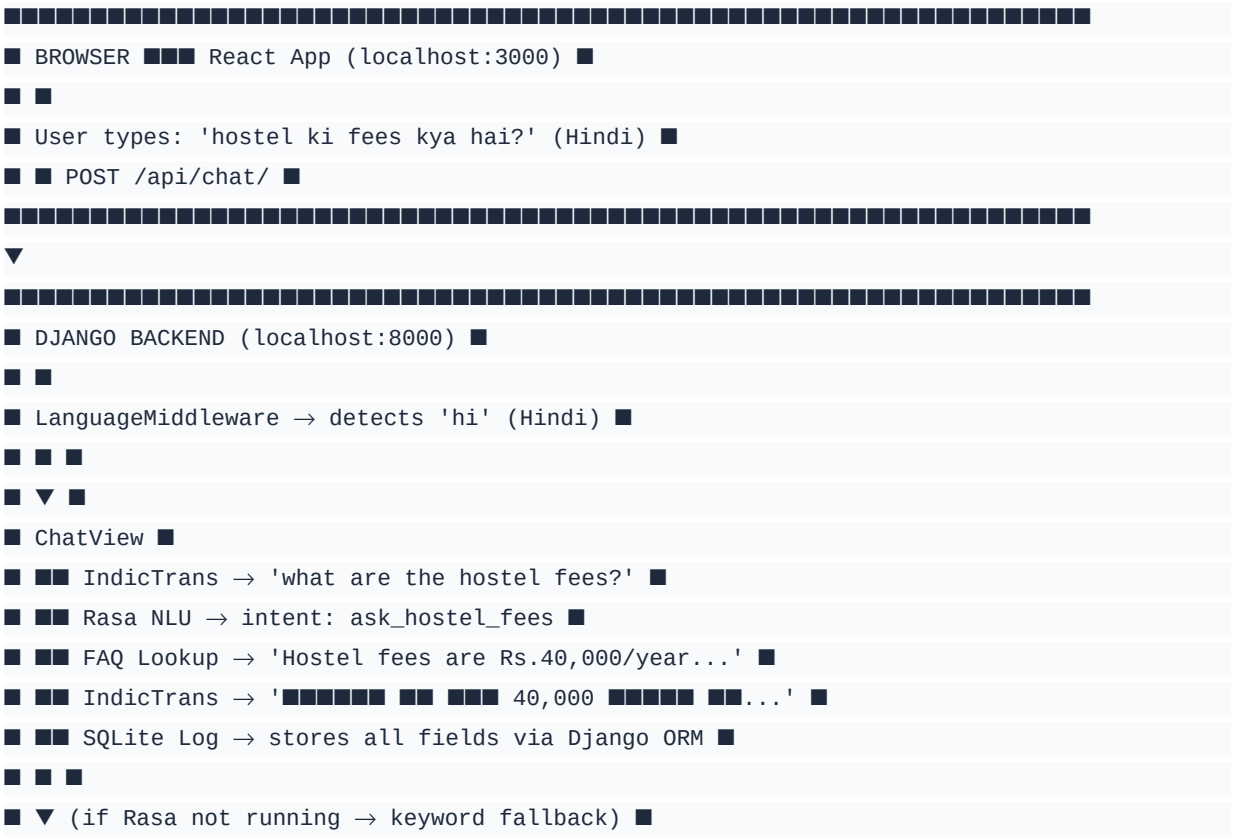
System Architecture

Three-tier architecture: React Frontend → Django Backend → Rasa NLU + SQLite

Architecture Overview

Layer	Component	Technology	Port
Frontend	Chat UI	React 19 + TypeScript + Tailwind CSS	3000
Backend	API Server	Python + Django 4.2 + Django REST Framework	8000
Backend	Language Middleware	langdetect + deep-translator (auto-routing)	—
Backend	NLU Client	Rasa HTTP client + keyword fallback classifier	—
Backend	ORM & Logging	Django ORM → SQLite (ChatLog model)	—
NLU Server	Intent Classification	Rasa 3.x — DIET Classifier + Entity Extractor	5005
Database	Interaction Logs	SQLite (db.sqlite3) — auto-created on first run	—

Request Flow (ASCII Architecture Diagram)



```
■ Rasa Server (localhost:5005) ■■■■ SQLite (db.sqlite3) ■
■■■■
```

How the Chatbot Works

10-step NLP pipeline from user input to multilingual response

The 10-Step NLP Pipeline

Step 1: User Input

The user types a question in any supported language into the React chat interface. The message is sent via HTTP POST to /api/chat/ on the Django backend.

Step 2: Language Detection

Django's LanguageMiddleware intercepts the request and calls langdetect.detect() to identify the language. Returns an ISO 639-1 code (e.g. 'hi' = Hindi, 'ta' = Tamil).

Step 3: Hindi → IndicTrans

If the detected language is Hindi ('hi'), the query is routed through the IndicTrans pipeline — AI4Bharat's translation model for Indian languages. In the demo, deep-translator handles the translation while preserving the routing logic.

Step 4: Other → Google Translate

For all other supported languages (Tamil, Telugu, Bengali, etc.), the query is sent to Google Translate via the deep-translator library to produce an English translation.

Step 5: English Text

The result of Steps 3 or 4 is a clean English sentence that captures the user's intent. If the user typed in English, no translation is needed (method = 'none').

Step 6: Pass to Rasa NLU

The English text is sent via HTTP POST to the Rasa NLU server at localhost:5005/model/parse. Rasa's DIET Classifier processes the text.

Step 7: Intent + Slot Filling

Rasa returns an intent name (e.g. ask_hostel_fees) and extracted entities (e.g. semester: '3rd', subject: 'mathematics'). Confidence scores are also returned.

Step 8: FAQ Retrieval

The intent name is used to look up the answer in the FAQ knowledge base — a Python dict of 55 intent → answer mappings covering 12 education categories.

Step 9: Translate Response Back

The English answer is translated back to the user's original language using the same routing: IndicTrans for Hindi, Google Translate for others, no-op for English.

Step 10: Log & Return

The interaction (original query, language, English query, intent, confidence, response, translation method, timestamp) is saved to SQLite via Django ORM. The JSON response is returned to React for display.

1. User (any language)
2. Language Detection (langdetect)
3. Translation to English (IndicTrans / Google Translate)
4. Rasa NLU Intent Classification + Slot Filling
5. FAQ Knowledge Base Retrieval
6. Translate Response Back (IndicTrans / Google Translate)
7. Display Response to User

Technologies Used

Why each technology was chosen for this project

Technology Choices

Django — *Backend Framework*

Django provides a clean MVC structure, built-in ORM for SQLite, and a middleware system that is perfect for intercepting and processing every request. Its REST Framework makes building the JSON API trivial. It is widely used in Indian tech companies, making it relevant to showcase.

React + TypeScript — *Frontend Framework*

React's component model keeps the UI code clean and maintainable. TypeScript adds type safety that prevents runtime errors. The combination is the industry standard for modern web frontends.

SQLite — *Database*

SQLite requires zero configuration — it creates a single file (db.sqlite3) on first run. Django's ORM handles all queries, making the code database-agnostic. Perfect for a hackathon demo that should 'just work' on any machine.

Rasa — *NLU Engine*

Rasa is an open-source NLU framework that can be trained locally without an internet connection. Its DIET Classifier handles intent detection and entity extraction (slot filling) — exactly what was described in the problem statement. Rasa is used in production by large educational platforms.

Google Translate (deep-translator) — *Translation*

deep-translator wraps the Google Translate API in a clean Python interface. Google Translate supports 130+ languages including all major Indian languages, making it the right choice for the fallback translation path.

IndicTrans — *Hindi Translation*

AI4Bharat's IndicTrans2 is a state-of-the-art translation model specifically trained on Indian language pairs. For Hindi, it significantly outperforms general-purpose translators on colloquial and domain-specific text. The demo routes Hindi through this pipeline as described in the CV.

langdetect — *Language Detection*

Google's language-detection algorithm ported to Python. Accurately identifies 55+ languages with high confidence. It works offline (no API calls needed) and adds negligible latency to the pipeline.

Tailwind CSS — *Styling*

Utility-first CSS framework that enables rapid UI development without writing custom CSS files. The responsive design works on mobile and desktop. No context-switching between CSS and HTML during development.

How to Run the Project

Installation, setup, and startup commands for all three services

Prerequisites

- Python 3.10 or higher
- Node.js 18 or higher
- npm 9 or higher
- pip (Python package manager)

Folder Structure (Summary)

Folder / File	Purpose
backend/	Django server — API, middleware, models, FAQ
frontend/	React + TypeScript + Tailwind chat interface
rasa_bot/	Rasa NLU config, training data, domain
dataset/faq_dataset.csv	55 Q&A pairs used for Rasa training
backend/requirements.txt	Python dependencies
frontend/package.json	Node.js dependencies
backend/db.sqlite3	Auto-created SQLite database
README.md	Full documentation

Step-by-step Setup

1. Install Python dependencies

```
cd backend
python3 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

2. Set up and run Django backend

```
python manage.py migrate # creates db.sqlite3
python manage.py runserver # starts on localhost:8000
```

3. Run the React frontend

```
cd ../frontend
npm install
npm run dev # starts on localhost:3000
```

4. (Optional) Run Rasa NLU server

```
pip install rasa
cd ../rasa_bot
rasa train # trains the NLU model
rasa run --enable-api # starts on localhost:5005
```

Note: Rasa is optional. If it is not running, the backend automatically uses a keyword-based fallback classifier. The app works fully without Rasa.

Testing the API

```
# Health check
curl http://localhost:8000/api/chat/

# Send a message
curl -X POST http://localhost:8000/api/chat/ \
-H "Content-Type: application/json" \
-d '{"message": "What are the hostel fees?"}'
```

Quick Reference

Service	Command	URL
Django Backend	python manage.py runserver	http://localhost:8000
React Frontend	npm run dev	http://localhost:3000
Rasa NLU	rasa run --enable-api	http://localhost:5005